

1. Title Page

Lab Name: Lab 12: Switched LANs, VLANs, and Inter-VLAN Routing **Date:** YYYY-MM-DD **Name:** [Your Name] **Lab Partners:** [Partners' Names, if any] **Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition.

2. Background

Local Area Networks (LANs) are often segmented using switches to improve performance and security. Virtual LANs (VLANs) logically separate a single physical switch into multiple distinct broadcast domains, isolating traffic between different groups of devices. To allow communication between these isolated VLANs, Inter-VLAN routing must be implemented using a router or a Layer 3 switch. In this lab, you will configure switched LANs and VLANs within your CML topology, and set up Inter-VLAN routing to control and facilitate communication between logically separated network segments.

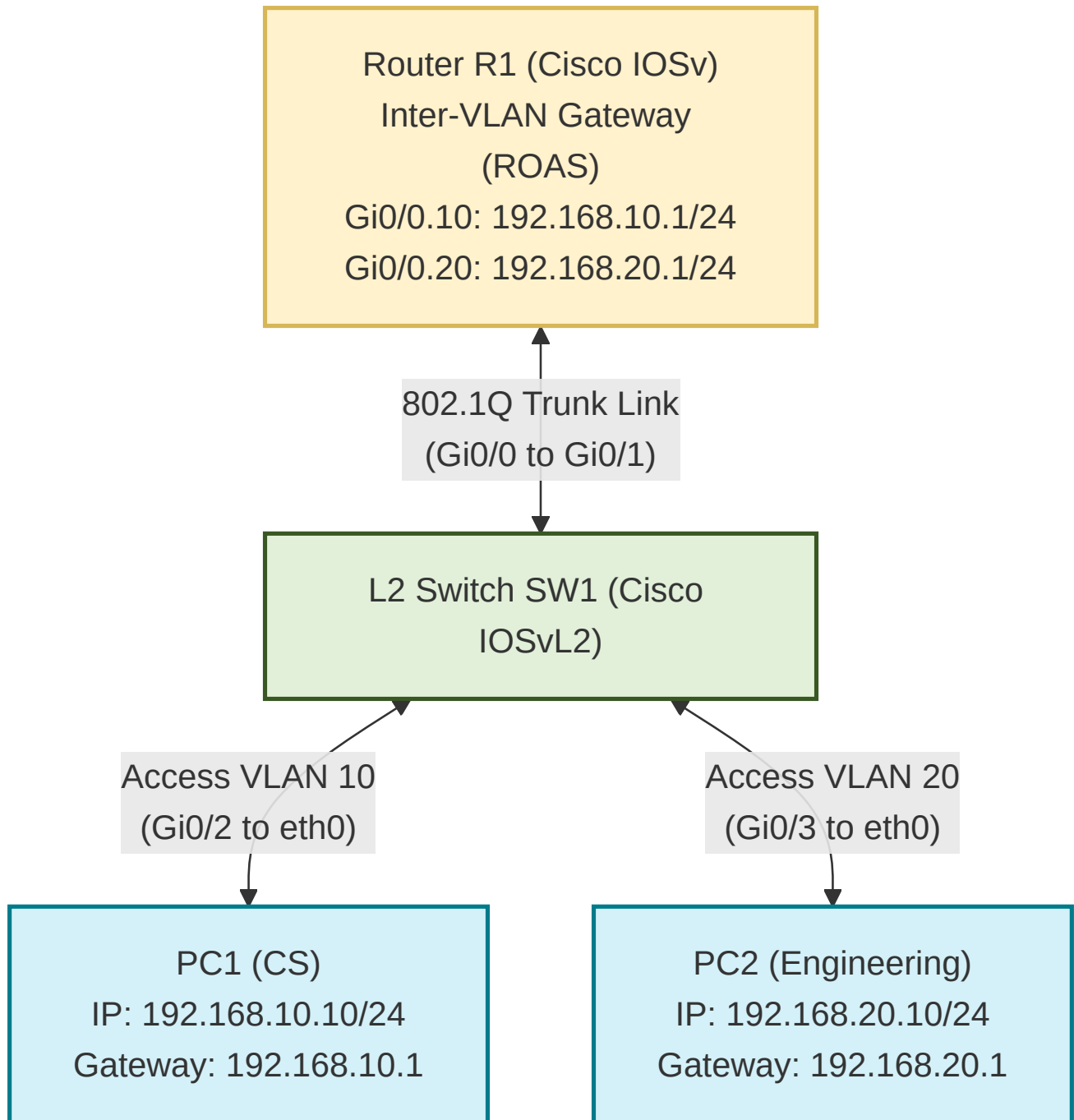
VLAN Tagging Concepts (IEEE 802.1Q)

- **What is VLAN Tagging?** Standard Ethernet frames generated by PCs and servers are **untagged**—they do not contain any information indicating which VLAN they belong to. To maintain isolation across a shared switch, the switch must track which VLAN each frame belongs to. Under the **IEEE 802.1Q** standard, a 4-byte VLAN tag is inserted into the Ethernet frame header. This tag contains a 12-bit **VLAN ID** (supporting VLANs 1-4094) that identifies the frame's broadcast domain.
 - **Access Ports vs. Trunk Ports:**
 - **Access Port:** A switch port assigned to a single VLAN.
 - *Ingress (Incoming):* When an untagged frame enters an access port, the switch internally assigns the port's configured VLAN ID to the frame.
 - *Egress (Outgoing):* When forwarding a frame out an access port to an end device (like a PC or a router interface in access mode), the switch strips off the VLAN tag so standard network cards can process the native Ethernet frame.
 - **Trunk Port:** A switch port configured to carry traffic for multiple VLANs simultaneously. Trunk ports preserve the 802.1Q tag on outgoing frames so the receiving switch or router knows which VLAN each frame belongs to.
-

3. Objective / Purpose

To segment a Layer 2 network using an IOSvL2 switch with VLANs and configure an IOSv router via a single 802.1Q trunk link (Router-on-a-Stick / ROAS) to route traffic between those VLANs. Separating traffic between the Computer Science (CS) and Engineering departments enhances network security by preventing unauthorized access to sensitive departmental data and intellectual property. Furthermore, isolating broadcast domains on the IOSvL2 switch reduces network congestion and enables controlled access policies via router subinterfaces over the trunk link.

4. Topology Diagram



Details:

- 1x IOSv Router (<initials>-R1), 1x IOSvL2 Switch (<initials>-SW1), 2x Linux Client Nodes (<initials>-PC1 , <initials>-PC2) in different VLANs.

5. Equipment & Environment

- Software:** Cisco Modeling Labs (CML), Terminal Emulator (e.g., PuTTY, SecureCRT, native terminal), Wireshark

- **Hardware / Virtual Nodes:** 1x IOSv Router (<initials>-R1), 1x IOSvL2 Switch (<initials>-SW1), 2x Linux Nodes (<initials>-PC1 , <initials>-PC2)

6. Configuration / Methodology

Note on Node Naming Convention: As established in Lab 1, set all node hostnames using your capitalized first and last initials in place of <initials> (for example, if your initials are KH, your nodes will be named KH-R1, KH-SW1, KH-PC1, and KH-PC2).

[!IMPORTANT] Interface & Port Mapping Warning: The interface port numbers used in the example commands below (such as Gi0/1 for the trunk link to the router, Gi0/2 for CS PC1, and Gi0/3 for Engineering PC2) reflect the sample topology diagram. Pay close attention to how you wire your nodes in CML! You **MUST** adjust the interface names in your switch (<initials>-SW1) and router (<initials>-R1) CLI commands to match the exact ports connected in your own CML topology workspace.

IP Addressing Table

Device	CML Node Name	Interface / VLAN	IP Address	Subnet Mask	Default Gateway
Client (CS)	<initials>-PC1	eth0 (VLAN 10)	192.168.10.10	255.255.255.0	192.168.10.1
Client (Engineering)	<initials>-PC2	eth0 (VLAN 20)	192.168.20.10	255.255.255.0	192.168.20.1
Router	<initials>-R1	Gi0/0.10 (VLAN 10 Gateway)	192.168.10.1	255.255.255.0	N/A
Router	<initials>-R1	Gi0/0.20 (VLAN 20 Gateway)	192.168.20.1	255.255.255.0	N/A

Switch Configuration (<initials>-SW1)

The switch performs Layer 2 segmentation by creating VLAN broadcast domains and managing port tagging:

1. **Hostname & VLAN Creation:** Set hostname (hostname <initials>-SW1) and define VLAN 10 (CS) and VLAN 20 (Engineering) in the switch database.
2. **Access Ports:** Configure client ports (Gi0/2 for CS and Gi0/3 for Engineering) in access mode (switchport mode access & switchport access vlan <ID>).
3. **Trunk Port:** Configure port Gi0/1 facing the router as an 802.1Q trunk link (switchport trunk encapsulation dot1q & switchport mode trunk).
4. **VLAN Status Verification:** Execute show vlan brief to display all active VLANs, their names, status (active), and assigned physical switch ports.

! Step 1: Set Hostname & Create VLAN Database Entries

```
SW1> enable
SW1# configure terminal
SW1(config)# hostname <initials>-SW1
```

```
KH-SW1(config)# vlan 10
KH-SW1(config-vlan)# name CS
```

```

KH-SW1(config-vlan)# exit

KH-SW1(config)# vlan 20
KH-SW1(config-vlan)# name Engineering
KH-SW1(config-vlan)# exit

! Step 2: Configure Host Access Ports
KH-SW1(config)# interface GigabitEthernet0/2
KH-SW1(config-if)# switchport mode access
KH-SW1(config-if)# switchport access vlan 10
KH-SW1(config-if)# exit

KH-SW1(config)# interface GigabitEthernet0/3
KH-SW1(config-if)# switchport mode access
KH-SW1(config-if)# switchport access vlan 20
KH-SW1(config-if)# exit

! Step 3: Configure 802.1Q Trunk Port to Router
KH-SW1(config)# interface GigabitEthernet0/1
KH-SW1(config-if)# switchport trunk encapsulation dot1q
KH-SW1(config-if)# switchport mode trunk
KH-SW1(config-if)# end

! Step 4: Verify VLAN Status and Assigned Ports
KH-SW1# show vlan brief
KH-SW1# show interface trunk

```

Router Configuration (<initials>-R1)

The router performs Inter-VLAN Layer 3 routing over the single trunk connection using logical subinterfaces:

1. **Hostname & Physical Interface:** Set hostname (`hostname <initials>-R1`) and enable physical interface `Gi0/0` (`no shutdown`).
2. **Logical Subinterfaces:** Create subinterfaces for each VLAN (`Gi0/0.10` and `Gi0/0.20`).
3. **802.1Q Encapsulation:** Configure `encapsulation dot1q <VLAN_ID>` on each subinterface to tag and route matching VLAN traffic.
4. **Gateway IP Addresses:** Assign the default gateway IP address for each subnet.
5. **Subinterface & Route Status Verification:** Execute `show ip interface brief` to verify subinterfaces `Gi0/0.10` and `Gi0/0.20` are `up/up` with assigned gateway IP addresses, and `show ip route` to confirm connected routes for both VLAN subnets.

```

! Step 1: Set Hostname & Enable Parent Physical Interface
Router> enable
Router# configure terminal
Router(config)# hostname <initials>-R1

KH-R1(config)# interface GigabitEthernet0/0
KH-R1(config-if)# no shutdown
KH-R1(config-if)# exit

```

! Step 2: Configure VLAN 10 Subinterface (CS Gateway)

```
KH-R1(config)# interface GigabitEthernet0/0.10
KH-R1(config-subif)# encapsulation dot1Q 10
KH-R1(config-subif)# ip address 192.168.10.1 255.255.255.0
KH-R1(config-subif)# exit
```

! Step 3: Configure VLAN 20 Subinterface (Engineering Gateway)

```
KH-R1(config)# interface GigabitEthernet0/0.20
KH-R1(config-subif)# encapsulation dot1Q 20
KH-R1(config-subif)# ip address 192.168.20.1 255.255.255.0
KH-R1(config-subif)# end
```

! Step 4: Verify Subinterface Status and Routing Table

```
KH-R1# show ip interface brief
KH-R1# show ip route
```

Narrative

Layer 2 switch broadcast domains on <initials>-SW1 are segmented into VLANs 10 (CS) and 20 (Engineering). Host ports Gi0/2 (<initials>-PC1) and Gi0/3 (<initials>-PC2) are configured as access ports for their respective VLANs. Inter-VLAN routing is accomplished over a single physical link (<initials>-SW1 Gi0/1 to <initials>-R1 Gi0/0) configured as an 802.1Q trunk port. Router <initials>-R1 uses logical subinterfaces (Gi0/0.10 and Gi0/0.20) with 802.1Q encapsulation to tag and route traffic between the subnets across the single trunk link.

7. Verification & Packet Capture Procedure

Step 1: CLI Verification Commands

1. Switch Verification:

- Open Switch <initials>-SW1 console and verify VLAN database entries:

```
KH-SW1# show vlan brief
```

- Verify the 802.1Q trunk port status on interface Gi0/1 :

```
KH-SW1# show interface trunk
```

2. Router Verification:

- Open Router <initials>-R1 console and verify subinterface routing entries:

```
KH-R1# show ip route
```

Step 2: Packet Capture Execution

You will capture traffic at **two distinct locations** in the network to observe how frames are processed on untagged access links versus tagged trunk links.

Capture Location 1: Host-to-Switch Access Link (<initials>-PC1 to <initials>-SW1)

- In the CML workspace, click on the link connecting <initials>-PC1 to <initials>-SW1 (Gi0/2).

2. Navigate to the **Packet Capture** tab in the bottom pane and click **Start**.
3. Open the console for host **<initials>-PC1** and ping both its default gateway and host **<initials>-PC2** :

```
ping -c 4 192.168.10.1
ping -c 4 192.168.20.10
```

4. Stop the capture in CML and download the packet capture file (`host_access_link.pcap`).

Capture Location 2: Switch-to-Router Trunk Link (**<initials>-SW1 to **<initials>-R1**)**

1. Click on the single 802.1Q trunk link connecting Switch **<initials>-SW1** (`Gi0/1`) to Router **<initials>-R1** (`Gi0/0`).
2. Navigate to the **Packet Capture** tab in the bottom pane and click **Start**.
3. From host **<initials>-PC1** , generate traffic across the trunk link to host **<initials>-PC2** :

```
ping -c 4 192.168.20.10
```

4. Stop the capture in CML and download the packet capture file (`switch_router_trunk.pcap`).

8. Wireshark Analysis and Questions

Open your downloaded packet captures (`host_access_link.pcap` and `switch_router_trunk.pcap`) in Wireshark and answer the following questions:

Part A: Host-to-Switch Access Link Capture (`host_access_link.pcap`)

1. Select an **ICMP Echo Request** frame sent from **<initials>-PC1** (`192.168.10.10`) destined for **<initials>-PC2** (`192.168.20.10`).
 - Inspect the packet details pane in Wireshark. Is an **802.1Q Virtual LAN** header present in this frame?
 - Explain why access link frames facing end host PCs do not contain 802.1Q VLAN tags.
2. Examine the Layer 2 Ethernet II frame header for this packet:
 - What is the destination MAC address of this frame? Does it match PC2's MAC address or Router R1's gateway interface MAC address?
 - Explain why PC1 sends frames destined for another subnet to the gateway MAC address rather than directly to PC2's MAC address.

Part B: Switch-to-Router Trunk Link Capture (`switch_router_trunk.pcap`)

3. Select an **ICMP Echo Request** frame traveling from **<initials>-SW1** to **<initials>-R1** on the trunk link (PC1 to PC2).
 - Locate the **IEEE 802.1Q Virtual LAN Info** header inserted between the Ethernet II header and the IP payload. What is the value of the **VLAN Identifier (VLAN ID)** field?
 - What hexadecimal value is displayed in the **Type** field of the 802.1Q header (`0x8100`)? What EtherType is listed inside the 802.1Q tag for the encapsulated payload?
4. Select the corresponding **ICMP Echo Reply** frame (or forwarded Echo Request) returning from Router **<initials>-R1** down the trunk link toward **<initials>-PC2** .
 - What is the **VLAN ID** in the 802.1Q header of this frame as it travels from the router back to the switch?
 - Trace the frame's complete journey: how does Router **<initials>-R1** receive a packet tagged with VLAN 10 on subinterface `Gi0/0.10` and send it back out on subinterface `Gi0/0.20` tagged with VLAN 20 over the same physical wire?

Part C: CLI & Inter-VLAN Routing Verification

5. Inspect the output of `show interface trunk` on Switch `<initials>-SW1` and `show ip route` on Router `<initials>-R1` :
- According to `show interface trunk` , which VLANs are allowed and active on port `Gi0/1` ?
 - According to `show ip route` , what subnets are directly connected to subinterfaces `Gi0/0.10` and `Gi0/0.20` ?
-

9. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything] **Discovery Method:** [How did you find the issue?]

Resolution: [How did you fix it?]

(If no issues were encountered, explain potential pitfalls for this configuration).

10. Conclusion & Analysis

Summarize your findings. Explain the structural difference between untagged Ethernet frames on switch access ports and 802.1Q tagged frames on trunk links, and describe how Router-on-a-Stick subinterfaces process VLAN tags to facilitate inter-VLAN routing across a single trunk link.

11. What to Hand In

- The Lab YAML configuration file with your name, date, and name of the lab at the top in comments.
- A single PDF report containing:
 - Your written answers to the 5 Wireshark and CLI analysis questions in Section 8.
 - A screenshot of your active CML topology showing all running nodes (`<initials>-R1` , `<initials>-SW1` , `<initials>-PC1` , `<initials>-PC2`).
 - A screenshot of the Switch CLI console displaying the output of `show vlan brief` confirming the active status of VLAN 10 (CS) and VLAN 20 (Engineering) with their assigned ports, as well as `show interface trunk` .
 - A screenshot of the Router CLI console displaying the output of `show ip interface brief` confirming subinterfaces `Gi0/0.10` and `Gi0/0.20` are up/up, and `show ip route` showing connected routes for both VLAN subnets.
 - An annotated Wireshark screenshot from `host_access_link.pcap` showing an **untagged** Ethernet frame on the PC-to-Switch access link.
 - An annotated Wireshark screenshot from `switch_router_trunk.pcap` highlighting the **IEEE 802.1Q Virtual LAN header** (with VLAN ID 10 and VLAN ID 20) on the Switch-to-Router trunk link.